MUHAMMAD ATHAR ABBAS
SP25 - BSE - 082
OBJECT ORIENTED PROGRAMMING
THEORY ASSIGNMENT # 2

Current Account class:

```
public class CurrentAccount{
   private int id;
   private double openingBalance;
   private String holderName;
   private int age;
   private boolean zakatOpt;
   private double balance = 0;

   public CurrentAccount(){};
   public CurrentAccount(int id, double openingBalance, String
name, int age, boolean zakatOpt){
       this.id = id;
       this.openingBalance = openingBalance;
       this.balance += openingBalance;
       this.holderName = name;
       this.age = age;
       this.zakatOpt = zakatOpt;
   }

   public static void main(String[] args) {
       CurrentAccount a = new CurrentAccount(202 , 13400, "Ajmal
Nasir" , 46, true);
       a.checkBalance();
```

```
        a.deposit(15000);
        a.withdraw(10000);
        a.deductZakat();
        a.checkBalance();
    }

    public void withdraw(double amount){
        System.out.println("\nWithdrawing money:");
        if(getBalance() > amount){
        balance -= amount;
        System.out.println("Amount: " + amount + " has been
withdrawn from your account.\n" + "Your Current Balance is " +
getBalance());
        } else {
            System.out.println("Insufficient funds");
        }
    }

    public void deposit(double amount){
        System.out.println("\nDepositing money:");
        balance += amount;
        System.out.println("Amount: " + amount + " has been
deposited in your account.\n");
        System.out.println("Current status:");
        checkBalance();
    }


    public void checkBalance(){
        System.out.println("\nBalance Checking:");
```

```
        System.out.println("Name : " + holderName + "\nAccount
Id: " +  id + "\nCurrent Balance: " + balance);
    }

    public double deductZakat(){
        System.out.println("\nDeducting Zakat.");
        if(zakatOpt){
            double zakat = EarningAccounts.bankersRound(balance *
0.025);
            balance -= zakat;
            return zakat;
        }
        return 0;
    }

    public int getId() {
        return id;
    }

    public void setId(int id) {
        this.id = id;
    }

    public double getOpeningBalance() {
        return openingBalance;
    }

    public void setOpeningBalance(double openingBalance) {
        this.openingBalance = openingBalance;
    }
```

```
public String getHolderName() {
    return holderName;
}

public void setHolderName(String holderName) {
    this.holderName = holderName;
}

public int getAge() {
    return age;
}

public void setAge(int age) {
    this.age = age;
}

public boolean isZakatOpt() {
    return zakatOpt;
}

public void setZakatOpt(boolean zakatOpt) {
    this.zakatOpt = zakatOpt;
}

public double getBalance() {
    return balance;
}

public void setBalance(double balance) {
```

```
        this.balance = balance;
    }
}
```

Output:

```
[athar@athar ~/semester2/OOP/java_programs/assignments]$ javac *.java
[athar@athar ~/semester2/OOP/java_programs/assignments]$ java CurrentAccount

Balance Checking:
Name : Ajmal Nasir
Account Id: 202
Current Balance: 13400.0

Depositing money:
Amount: 15000.0 has been deposited in your account.

Current status:

Balance Checking:
Name : Ajmal Nasir
Account Id: 202
Current Balance: 28400.0

Withdrawing money:
Amount: 10000.0 has been withdrawn from your account.
Your Current Balance is 18400.0

Deducting Zakat.

Balance Checking:
Name : Ajmal Nasir
Account Id: 202
Current Balance: 17940.0
[athar@athar ~/semester2/OOP/java_programs/assignments]$
```

Earning Accounts class:

Both earning accounts , investment and saving accounts inherit this class.

```
import java.math.BigDecimal;
import java.math.RoundingMode; // for banking systems round off
public class EarningAccounts extends CurrentAccount{
   private boolean filer;

   public EarningAccounts(int id, double openingBalance, String
name, int age, boolean zakatOpt, boolean filer){
       super(id,openingBalance,name,age,zakatOpt);
       this.filer = filer;
   }

   public double calculateEarnings(){
       return 0.0;
   };

   public static double getTotalProfitPaid(EarningAccounts...
records){
       double liabilities = 0.0;
       for(EarningAccounts account: records){
           liabilities += account.calculateEarnings();
       }
       return bankersRound(liabilities);
   }

   public void reinvest(){
       System.out.println("\nReinvesting money:");
       double earnings = bankersRound(calculateEarnings());
       System.out.println("Current Earnings " + earnings + " are
reinvested.");
       setBalance(bankersRound(getBalance() + earnings));
```

```
        System.out.println("Your current balance is : " +
getBalance());
    }

    //to round off numbers by banking conventions. (AI written)
    public static double bankersRound(double value) {
    BigDecimal bd = new BigDecimal(String.valueOf(value));
    BigDecimal roundedBd = bd.setScale(2,
RoundingMode.HALF_EVEN);
    return roundedBd.doubleValue();
    }

    public boolean isfiler() {
        return filer;
    }

    public void setfiler(boolean filer) {
        this.filer = filer;
    }
}
```

(earnings are only added to the balance at the time of withdrawal. At time of balance checking they are shown separately.)

Saving Accounts class:

```
public class SavingAccount extends EarningAccounts{
    private enum Category {YOUNGSAVER(0.06), ADULTSAVER(0.07),
SENIORCITIZENSAVER(0.08);
        private final double ratio;
        private Category(double ratio) {
```

```
            this.ratio = ratio;
        }
        public double getratio() {
            return ratio;
        }
    };

    private final Category accountCategory;
    public SavingAccount(int id, double openingBalance, String
name, int age, boolean zakatOpt,boolean filer){
        super(id,openingBalance,name,age,zakatOpt,filer);
        if (getAge() < 35){
            accountCategory = Category.YOUNGSAVER;
        } else if (getAge() ⩾ 35 && getAge() ⩽ 50){
            accountCategory = Category.ADULTSAVER;
        } else {
            accountCategory = Category.SENIORCITIZENSAVER;
        }
    }

    @Override
    public double calculateEarnings(){
        double saving = (accountCategory.getratio() *
getBalance());
        if(isfiler()){
        saving -= (saving * 0.15); //capital gain taxes;
        } else {
            saving -= (saving * 0.25);
        }
        return bankersRound(saving);
```

```
    }

    @Override
    public void withdraw(double amount){
        System.out.println("\nWithdrawing money:");
        double profit = calculateEarnings();
        double totalBalance = getBalance() + profit;
        if(amount > totalBalance){
            System.out.println("Insufficient funds");
        } else {
            setBalance(totalBalance);
            double filerTax;
            String status;
            if(isfiler()){
                filerTax = bankersRound(profit * 0.02);
                status = "Filer";
            } else {
                filerTax = bankersRound(profit * 0.04);
                status = "Non-filer";
            }
            setBalance(bankersRound(getBalance() - (amount +
filerTax)));
            String message = String.format(
            "Name: %s\nAccount Id: %s\nAmount withdrawn:
%.2f\nWithdrawal Tax(%s): %.2f\nCurrent Balance: %.2f\n",
            getHolderName(),
            getId(),
            amount,
            status,
            filerTax,
```

```
            getBalance());
            System.out.printf(message);
        }
    }

    @Override
    public void checkBalance(){
        System.out.println("\nBalance Checking:");
        System.out.println("Name : " + getHolderName() +
"\nAccount Id: " + getId() + "\nCurrent Balance: " +
getBalance() + "\nSavings: " + calculateEarnings());
    }

    public static void main(String[] args) {
        SavingAccount s1 = new SavingAccount(99,145000,"Kazim
Shah", 76, true, true);
        s1.checkBalance();
        System.out.println("\nCurrent Earnings: " +
s1.calculateEarnings());
        s1.deductZakat();
        s1.withdraw(34000);
    }
}
```

Output:

```
[athar@athar ~/semester2/OOP/java_programs/assignments]$ java SavingAccount

Balance Checking:
Name : Kazim Shah
Account Id: 99
Current Balance: 145000.0
Savings: 9860.0

Current Earnings: 9860.0

Deducting Zakat.

Withdrawing money:
Name: Kazim Shah
Account Id: 99
Amount withdrawn: 34000.00
Withdrawal Tax(Filer): 192.27
Current Balance: 116796.23
[athar@athar ~/semester2/OOP/java_programs/assignments]$
```

Investment Accounts class:

```
public class InvestmentAccount extends EarningAccounts{


   private enum plans{ONEYEARPLAN(1,0.1), THREEYEARPLAN(3,0.12),
FIVEYEARPLAN(5,0.14);
       private final double ratio;
       private final int years;
       private plans(int years , double ratio) {
           this.years = years;
           this.ratio = ratio;
       }
       public double getRatio(){
           return this.ratio;
       }
```

```
        public int getYears() {
            return years;
        }
    };
    private plans investmentPlan;
    private Countries country;
    private int yearAccountOpened;
    //average inflation rates of different countries from
previous 5 years.
    private enum Countries{
            PAKISTAN("pakistan" , 0.1650, 0.1650), CHINA("china",
0.0116, 0.15), LEBANON("lebanon", 1.3548,0.15), JAPAN("japan" ,
0.0167,0.20), USA("usa", 0.0420,0.20) , UK("uk", 0.0430,0.20),
FRANCE("france",0.0284,0.30);
            private final String country;
            private final double inflationRate;
            private final double capitalGainTax;
            private Countries(String country, double rate, double
tax) {
            inflationRate = rate;
            this.country = country;
            this.capitalGainTax = tax;
            }

            public String getCountryName() {
                return country;
            }

            public double getInflationRate(){
```

```
            return inflationRate;
        }

        public double getCapitalGainTax() {
        return capitalGainTax;
        }
    }


    public InvestmentAccount(int id, double openingBalance,
String name, int age, boolean zakatOpt,boolean filer, int plan){
        super(id,openingBalance,name,age,zakatOpt,filer);
        switch (plan) {
            case 1:
                investmentPlan = plans.ONEYEARPLAN;
                break;
            case 3:
                investmentPlan = plans.THREEYEARPLAN;
                break;
            case 5:
                investmentPlan = plans.FIVEYEARPLAN;
            default:
                break;
        }
    }

    public InvestmentAccount(int id, double openingBalance,
String name, int age, boolean zakatOpt,boolean filer, int plan,
int year , String countryName){
    this(id,openingBalance,name,age,zakatOpt,filer,plan);
```

```
        switch (countryName) {
            case "pakistan":
                this.country = Countries.PAKISTAN;
                break;
            case "china":
                this.country = Countries.CHINA;
                break;
            case "lebanon":
                this.country = Countries.LEBANON;
                break;
            case "japan":
                this.country = Countries.JAPAN;
                break;
            case "usa":
                this.country = Countries.USA;
                break;
            case "uk":
                this.country = Countries.UK;
                break;
            case "france":
                this.country = Countries.FRANCE;
                break;
            default:
                this.country = null;
                break;
        }
            yearAccountOpened = year;
    }
```

```
@Override
public double calculateEarnings(){
    double currentBalance = getBalance();
    double profit = 0.0;
    double profitRate = investmentPlan.getRatio();
    for(int i = 1 ; i ⩽ investmentPlan.getYears();i++){
        profit += currentBalance * profitRate;
        currentBalance += profit;
    }
    if(isfiler()){
    profit -= (profit * 0.15); //capital gain taxes;
    } else {
        profit -= (profit * 0.25);
    }
    return bankersRound(profit);
}


public double getRealProfitRatio(){
    double currentBalance = getBalance();
    double totalProfit = 0.0;
    double profitRate = investmentPlan.getRatio();
    int yearPlan = investmentPlan.getYears();
    for(int i = 1 ; i ⩽ yearPlan ;i++){
        double yearlyProfit = currentBalance * profitRate;
        currentBalance += yearlyProfit;
        totalProfit += yearlyProfit;
    }
    totalProfit -= totalProfit * country.getCapitalGainTax();
    //inflation adjustment
```

```
        /*multiplicative factor is the factor by which the price
of something will change by inflation rate,
        in base of pow function we have the inflation factor of
price for 1 year ,
        to scale it to other years , we have that number in
exponent of pow.*/
        double multiplicativeFactor = Math.pow((1 +
country.getInflationRate()),yearPlan);
        double realReturn = totalProfit / multiplicativeFactor;
        return bankersRound(realReturn);
    }

    @Override
    public void withdraw(double amount){
        System.out.println("\nWithdrawing money:");
        double profit = calculateEarnings();
        double totalBalance = getBalance() + profit;
        if(amount > totalBalance){
            System.out.println("Insufficient funds");
        } else {
            setBalance(totalBalance);
            double filerTax;
            String status;
            if(isfiler()){
                filerTax = bankersRound(profit * 0.02);
                status = "Filer";
            } else {
                filerTax = bankersRound(profit * 0.04);
                status = "Non-filer";
            }
```

```
            double surcharge = bankersRound(amount * 0.04);
            setBalance(getBalance() - (amount + filerTax +
surcharge));
            String message = String.format(
            "Name: %s\nAccount Id: %s\nAmount withdrawn:
%.2f\nWithdrawal Tax(%s): %.2f\nSurcharge: %.2f\nCurrent
Balance: %.2f\n",
            getHolderName(),
            getId(),
            amount,
            status,
            filerTax,
            surcharge,
            getBalance());
            System.out.printf(message);
        }
    }

    @Override
    public void checkBalance(){
        System.out.println("\nBalance Checking:");
        System.out.println("Name : " + getHolderName() +
"\nAccount Id: " + getId() + "\nCurrent Balance: " +
getBalance() + "\nProfit: " + calculateEarnings());
    }

    public double getProfitRatio(){
        return investmentPlan.getRatio();
    }
    public String getCountryName(){
```

```
        return country.getCountryName();
    }
    public static void main(String[] args) {

        InvestmentAccount a = new InvestmentAccount(901, 1560000,
"Hisham Khalil" , 76, true, true, 3);
        InvestmentAccount a2 = new InvestmentAccount(907, 89000,
"Jasim Naeem" , 34, true, false,1);

        a.checkBalance();
        System.out.println("\nCurrent Earnings: " +
a.calculateEarnings());
        a.withdraw(56000);
        System.out.println();
        a.reinvest();
        //total Money paid by the bank
        System.out.println("\nTotal Profit paid by the bank: " +
getTotalProfitPaid(a,a2));



        //for checking highest return , --the extra feature.
       // int id, double openingBalance, String name, int age,
boolean zakatOpt,boolean filer, int plan, int year , String
countryName
        InvestmentAccount pak = new
InvestmentAccount(109,40000,"athar",49,false,false,5,2018,"pakis
tan");
```

```
        InvestmentAccount usa = new
InvestmentAccount(109,40000,"athar",49,false,false,5,2018,"usa")
;
        InvestmentAccount uk =  new
InvestmentAccount(109,40000,"athar",49,false,false,5,2018,"uk");
        InvestmentAccount lebanon = new
InvestmentAccount(109,40000,"athar",49,false,false,5,2018,"leban
on");
        InvestmentAccount france = new
InvestmentAccount(109,40000,"athar",49,false,false,5,2018,"franc
e");
        InvestmentAccount china =  new
InvestmentAccount(109,40000,"athar",49,false,false,5,2018,"china
");
        InvestmentAccount japan =  new
InvestmentAccount(109,40000,"athar",49,false,false,5,2018,"japan
");

        InvestmentAccount[] accounts = new
InvestmentAccount[]{pak,usa,uk,lebanon,france,china,japan};
        checkHighestReturn(accounts);
    }

    public static void checkHighestReturn(InvestmentAccount[]
collection){
        double highestReturns = 0;
        String countryName = "";
            for(InvestmentAccount acc : collection){
                double returns = acc.getRealProfitRatio();
                if(returns > highestReturns){
```

```
                    highestReturns = returns;
                    countryName = acc.getCountryName();
                }
        }
        System.out.println("\nHighest return: "+ highestReturns +
"  is by country: " + countryName + " (over investement of
40000).");
    }

    public int getYearAccountOpened() {
        return yearAccountOpened;
    }

    public void setYearAccountOpened(int yearAccountOpened) {
        this.yearAccountOpened = yearAccountOpened;
    }
}
```

Output:

```
[athar@athar ~/semester2/OOP/java_programs/assignments]$ java InvestmentAccount

Balance Checking:
Name : Hisham Khalil
Account Id: 901
Current Balance: 1560000.0
Profit: 556028.93

Current Earnings: 556028.93

Withdrawing money:
Name: Hisham Khalil
Account Id: 901
Amount withdrawn: 56000.00
Withdrawal Tax(Filer): 11120.58
Surcharge: 2240.00
Current Balance: 2046668.35


Reinvesting money:
Current Earnings 729491.54 are reinvested.
Your current balance is : 2776159.89

Total Profit paid by the bank: 996178.34

Highest return: 29701.01  is by country: china (over investement of 40000).
[athar@athar ~/semester2/OOP/java_programs/assignments]$
```

==========END==========